

EXAMEN D'ALGO-SDA - L3I

Exercice 1 : listes doublement chaînées (4.5 Pts)

Pour faciliter la gestion d'une liste doublement chaînée, on rajoutera deux adresses : la première adresse pour indiquer la tête de la liste et la seconde adresse pour indiquer la queue de la liste.

```
typedef int TElement;
typedef struct cellule{
    TElement donnee;
    struct cellule *suivant;
    struct cellule *precedent;
} *DListe;

typedef struct {
    DListe tete;
    DListe queue;
} DListeTQ;
```

Pour l'écriture des algorithmes ci-dessous, on suppose que la liste « DListeTQ » est triée en ordre croissant « <= ».

- 1) En parcourant la liste dans les deux sens ; écrire en C ; l'algorithme qui détermine l'adresse d'insertion d'un élément donné dans une liste non vide donnée.
- 2) Ecrire en C, l'algorithme d'insertion triée d'un élément donné dans une liste donnée.
- 3) En parcourant la liste dans les deux sens ; écrire en C, l'algorithme de suppression d'un élément donné dans une liste non vide donnée.
- 4) En parcourant la liste triée dans les deux sens ; écrire en C, l'algorithme de suppression d'une liste non vide donnée.

EXERCICE 2 : CODAGE DE HOFFMANN (6.5PTS)

On souhaite compresser un texte composé uniquement de 4 caractères d'alphabet A={a, b, c, d}, et dont les fréquences d'apparition des caractères de A sont : $frec(a)=120$, $frec(b)=40$, $frec(c)=90$ et $frec(d)=10$.

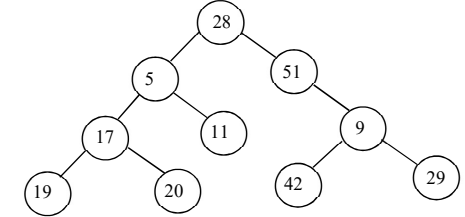
Pour faciliter l'écriture de certains algorithmes, on suppose que l'ensemble des caractères de l'alphabet $n=card(A)$ sont tous fréquents.

- 1) Décrire les structures de données utilisées pour implémenter la création de l'arbre de Hoffmann d'une part et pour décompresser le texte d'autre part (Justifier votre réponse).
- 2) Ecrire en C, la structure de donnée intermédiaire utilisée pour la création de l'arbre Hoffmann ? (Justifier votre réponse).
- 3) Ecrire en langage algorithmique (ou en langage C), l'algorithme de construction de l'arbre de Hoffmann à partir d'un texte donné par sa table de fréquence. (On suppose que les opérations de base de la structure intermédiaire sont définies dans un module donné (voir annexe)).
- 4) Appliquer pas à pas votre algorithme au texte donné par sa table de fréquence ci-dessous. C'est à dire décrire le contenu des différentes structures utilisées (principales et intermédiaire) pendant la phase d'initialisation et pendant chaque itération.
- 5) En déduire graphiquement l'arbre de Hoffmann (Justifier votre réponse).
- 6) Ecrire en langage algorithmique (ou en langage C), un algorithme qui calcule la hauteur de l'arbre de Hoffmann non vide donné décrite **uniquement** par le tableau Père.

Exercice 3 : AVL (5.5 Pts)

Note : on suppose connu l'algorithme d'équilibrage (voir annexe).
On définit la structure de données d'un AVL comme suit :

```
typedef int TElement;
typedef struct noeud{
    int hauteur;
    TElement donnee;
    struct noeud *filsG;
    struct noeud *filsD;
} *AVL;
```



- 1) Ecrire en C, l'algorithme d'insertion d'un élément donné dans un AVL donné.
- 2) Ecrire en langage algorithmique (ou en C), l'algorithme de construction d'un AVL à partir d'un arbre binaire donné parcouru selon le **parcours postfixe**.
- 3) Appliquer graphiquement (sous forme d'arbres) votre algorithme de construction (question 2) de l'AVL à partir de l'arbre binaire ci-dessus.
- 4) Ecrire en langage algorithmique ou en C, un algorithme qui vérifie l'égalité entre deux AVL donnés. Deux AVL sont identiques s'ils ont une structure identique et si leurs contenus est également le même.
- 5) Ecrire en C, un algorithme qui calcule dans un AVL donné, la somme des données associés aux différents nœuds qui sont supérieurs strictement à un élément donné.

Exercice 4 : ARBRE 2-3-4 (3.5 Pts)



```
#define NBMAXDONNEES 3
#define NBMAXFILS 4
typedef int TElement;
typedef struct nœud *Arbres234;
struct nœud{
    int estFeuille; //indicateur si un nœud est une feuille ou pas
    int nbFils; //le nombre de fils présents par nœud
    TElement donnee[NBMAXDONNEES]; // données associées à chaque nœud
    Arbres234 fils[NBMAXFILS]; //fils associés à chaque nœud
}
```

- 1) Insérer l'élément 33 dans l'arbre 2-3-4 ci-dessus.
- 2) Ecrire en langage algorithmique ou en C, un algorithme qui détermine l'adresse d'un élément donné dans un arbre 2-3-4 donné.
- 3) En effectuant le parcours en largeur dans un arbre 2-3-4 non vide donné ; écrire en C, un algorithme qui calcule le nombre d'occurrences de chaque type de nœuds. On suppose connues les différentes primitives de la structure intermédiaire utilisée (voir annexe)

NOTES

- ◆ Les réponses aux questions doivent être claires et justifiées.
- ◆ Pour chaque algorithme (ou fonction en C), vous aurez soin d'expliquer les différentes actions de l'algorithme les préconditions, ainsi que le passage des paramètres.
- ◆ **Chaque algorithme (ou fonction en C) supplémentaire utilisé doit être défini.**

*****Annexes*****

On suppose connu les algorithmes suivants :

Exercice 1 : DListeTQ

```
Fonction donnee : DListe → TElement
Fonction suivant : DListe → DListe
Fonction precedent : DListe → DListe
Fonction initDL : [] → DListe
Fonction estVideDL : DListe → Booléenne
Fonction tete : DListeTQ → DListe
Fonction queue : DListeTQ → DListe
Fonction initDLTQ : [] → DListeTQ
Fonction estVideDLTQ : DListeTQ → Booléenne
//Insertion en tête dans une liste vide
Fonction inserTeteLV : TElementxDListeTQ → DListeTQ
//Insertion en tête dans une liste non vide
Fonction inserTeteLNV : TElementxDListeTQ → DListeTQ
//Insertion en queue dans une liste non vide
Fonction inserQueue : TElementxDListeTQ → DListeTQ
Fonction suppTete : DListeTQ → DListeTQ
Fonction suppQueue : DListeTQ → DListeTQ
```

Exercice 2 : TasMin

```
TElement minTas(Tas tas) ;
Tas inserTas(TElement elt, Tas tas, int * freq) ;
Tas suppTas(Tas tas, int * freq) ;
```

Exercice 3 : AVL

```
Fonction donnee : AVL → TElement
Fonction hauteur : AVL → Entier
Fonction filsGauche : AVL → AVL
Fonction filsDroit : AVL → AVL
Fonction initA : [] → AVL
Fonction estVideA : AVL → Booléenne
Fonction equilibrage : AVL → AVL
```

Exercice 4 : Structure Arbres234

```
Fonction estFeuille : Arbres234 → Booléenne
Fonction nbFils : Arbres234 → Entier
Fonction iEmeDonnee : Entier x Arbres234 → TElement
Fonction iEmeFils : Entier x Arbres234 → Arbre234
```

Structure File

```
Fonction sommetF : File → TElementF
Fonction enfiler : TElementF x File → File
Fonction defiler : File → File
Fonction initF : [] → File
Fonction estVideF : File → Booléenne
Fonction estPleinF : File → Booléenne
```

Structure Pile

```
Fonction sommetP : Pile → TElementP
Fonction empiler : TElementPx Pile → Pile
Fonction depiler : Pile → Pile
Fonction initP : [] → Pile
Fonction estVideP : Pile → Booléenne
Fonction estPleinP : Pile → Booléenne
```